

ChatGPT

Tuesday 5:11 PM

I am doing a lab which involves building a web app (a todo list app), so I already have the functionality for adding a task, showing all tasks. The main thing about this is that it doesn't require anyone to login and it is a local app (using react & next.js) when it comes to editing a route, it clearly has to be dynamic such that we edit the task that the user selected, would it be smart to send every request with an id so that it will be easier for me to select that specific task from the database and edit it or is there a much more efficient and safer way

For your specific lab (a local todo app with no authentication/users), using the task ID in the route is exactly the right approach. In fact, it is the standard RESTful way to handle this.

The important distinction is: do not send the ID everywhere unnecessarily — use it where you are referring to a specific resource.

The usual pattern

Your routes would look something like:

GET /api/tasks
POST /api/tasks
...
+ Ask anything

ChatGPT

The usual pattern

Your routes would look something like:

GET /api/tasks
POST /api/tasks
GET /api/tasks/:id
PUT /api/tasks/:id
PATCH /api/tasks/:id

For editing:

/manage-task/edit/5

means:

"Open the edit page for the task whose id is 5."

Your Next.js structure would be:

app/
└─ manage-task/
 └─ edit/
 └─ [id]/
 └─ page.js

The `[id]` is a dynamic route.

So if the user clicks:

+ Ask anything

ChatGPT

So if the user clicks:

Edit

on:

Task:
Study React
Id: 7

you navigate to:

/manage-task/edit/7

Then inside `page.js` you get:

JavaScript
const { id } = params;

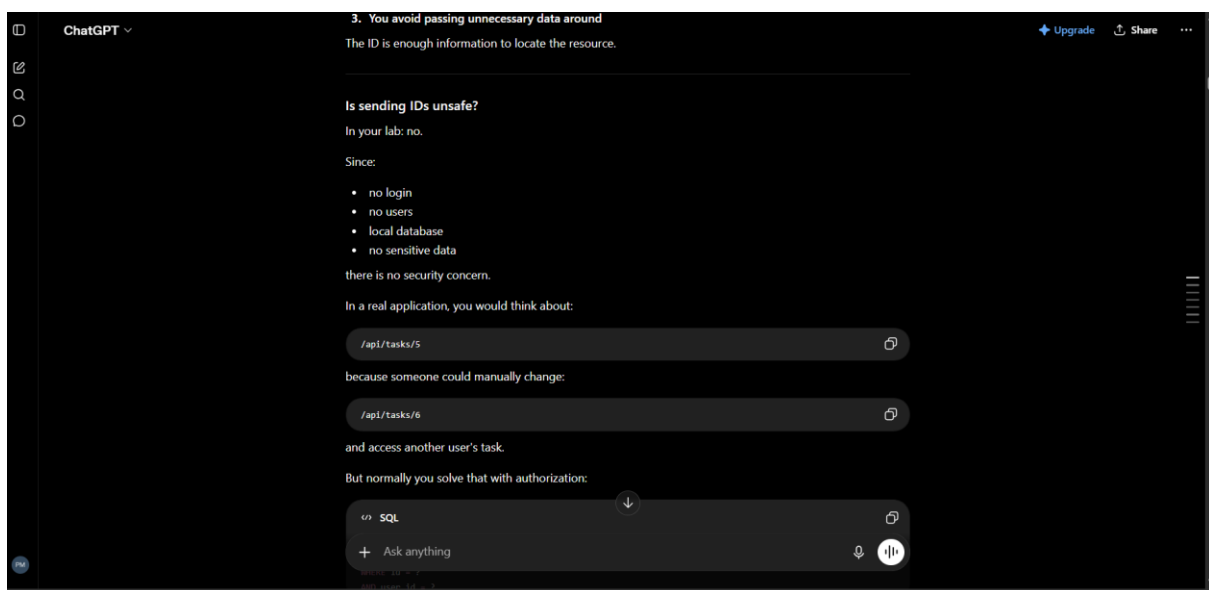
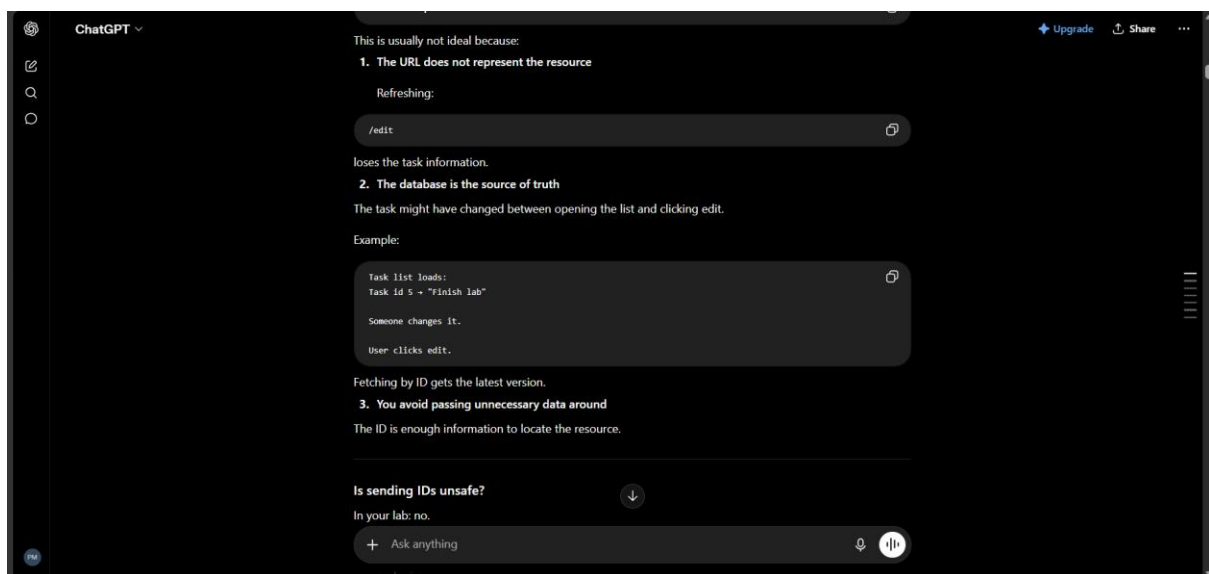
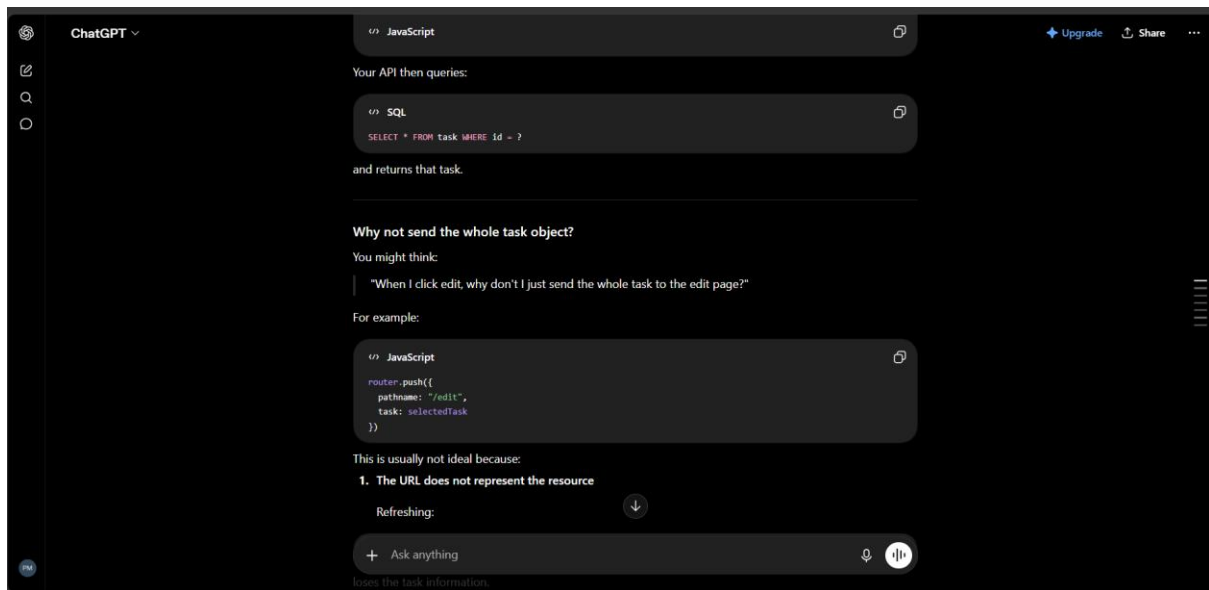
and fetch:

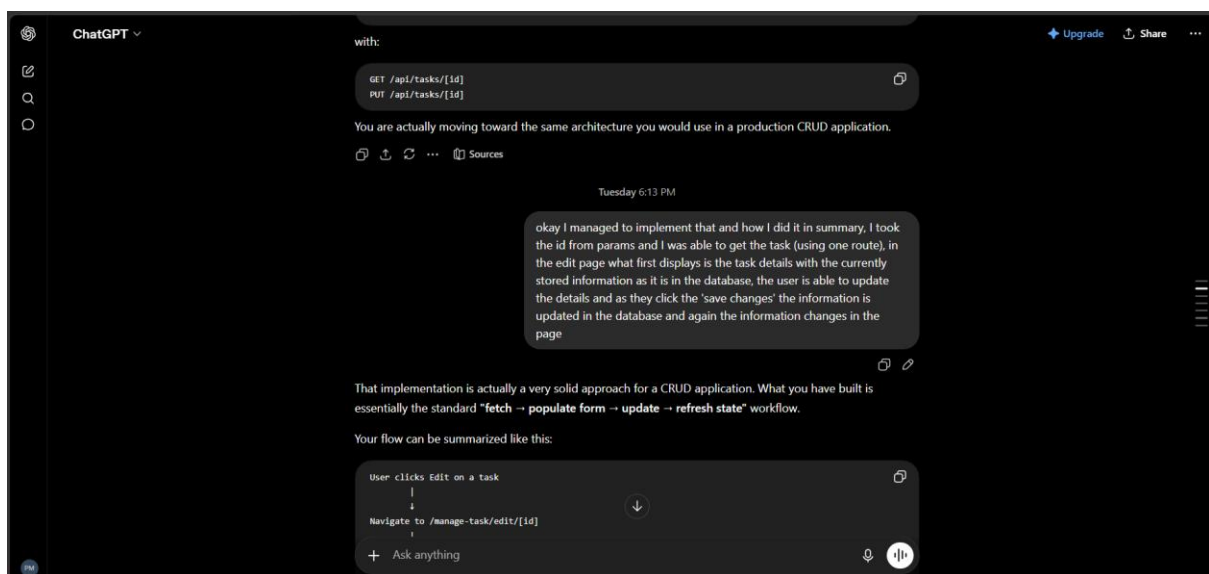
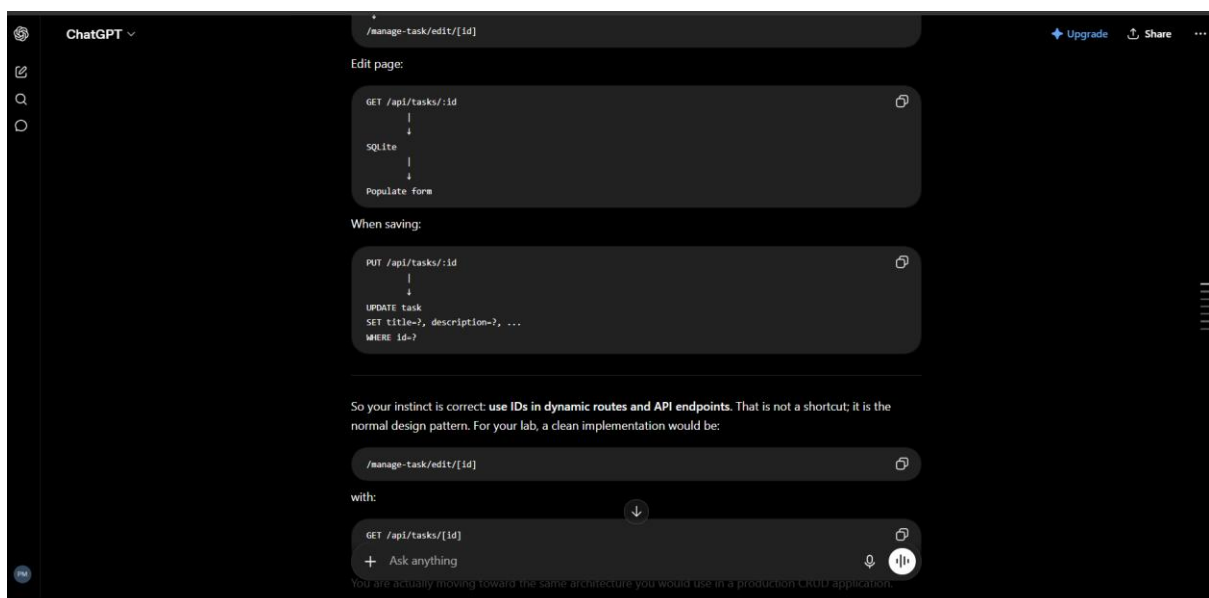
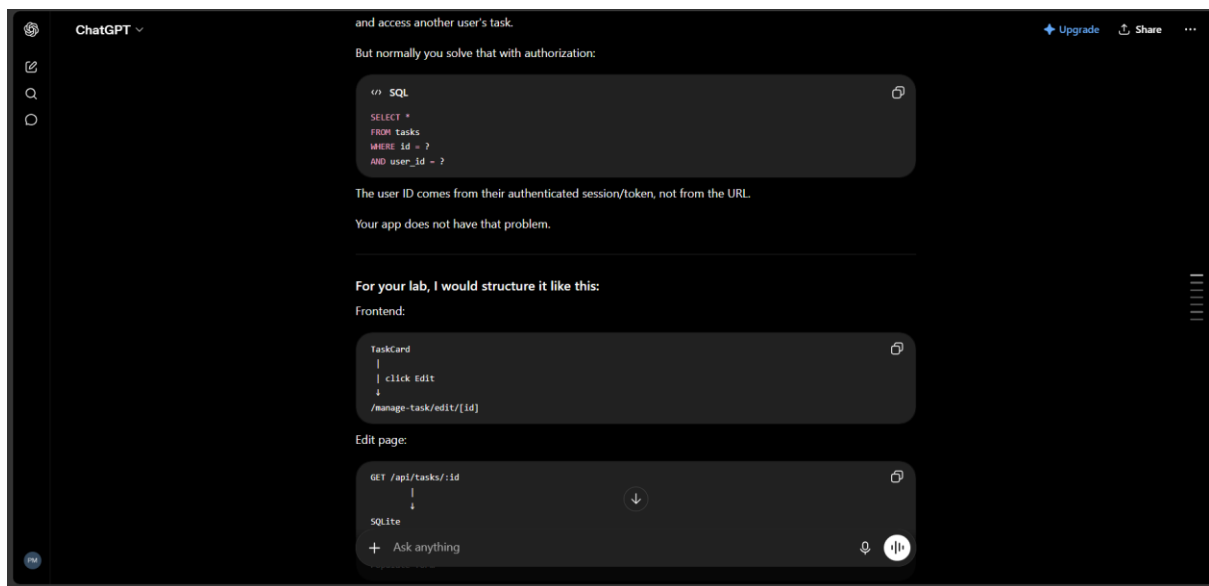
JavaScript
fetch(`/api/tasks/\${id}`)

Your API then queries:

SQL
SELECT * FROM task WHERE Id = ?

+ Ask anything





ChatGPT

That implementation is actually a very solid approach for a CRUD application. What you have built is essentially the standard "fetch → populate form → update → refresh state" workflow.

Your flow can be summarized like this:

```
graph TD
    A[User clicks Edit on a task] --> B[Navigate to /manage-task/edit/{id}]
    B --> C[Get id from params]
    C --> D[GET /api/tasks/{id}]
    D --> E[Query SQLite using id]
    E --> F[Return selected task]
    F --> G[Display current task details in form]
    G --> H[User modifies fields]
    H --> I[Click "Save Changes"]
    I --> J[Ask anything]
```

PM

ChatGPT

```
graph TD
    A[Click "Save Changes"] --> B[PUT /api/tasks/{id}]
    B --> C[UPDATE task in SQLite]
    C --> D[Fetch updated task / update state]
    D --> E[Display new information]
```

A few things you did correctly:

1. The ID is the link between the UI and database

Your edit page does not need to know the whole task initially. It only needs:

```
JavaScript
id = 5
```

From that, the backend knows:

```
SQL
SELECT *
FROM task
WHERE id = 5;
```

PM

ChatGPT

```
SQL
WHERE id = 5;
```

That keeps your frontend and backend nicely separated.

2. Loading the existing values first is the correct UX

This part:

"the edit page first displays the task details with the currently stored information"

is exactly how most editing interfaces work.

For example:

```
Title:
[Finish COMS3011A Lab]

Description:
[Implement edit route]

Due date:
[2025-08-04]
```

The user is not creating a new task; they are modifying an existing resource.

3. Updating the database instead of only changing the page is important

A common beginner mistake is:

PM

ChatGPT

Upgrade

Share

3. Updating the database instead of only changing the page is important

A common beginner mistake is:

JavaScript

```
setTitle(newTitle)
```

and thinking the task is edited.

But that only changes React state. Refresh the page and the change disappears.

You correctly did:

React state

↓

API request

↓

SQLite UPDATE

↓

Database permanently changed

That is the important part.

One small improvement you could consider (if you haven't already done it):

Instead of:

Save button clicked

↓

+ Ask anything

ChatGPT

Upgrade

Share

One small improvement you could consider (if you haven't already done it):

Instead of:

Save button clicked

↓

PUT request

↓

Fetch task again

↓

Display updated task

you can also update your local state using the response from the PUT request:

Example:

JavaScript

```
const response = await fetch(`/api/tasks/${id}`, {
  method: "PUT",
  body: JSON.stringify(updatedTask)
});

const updated = await response.json();
setTask(updated);
```

Your API returns the updated task:

+ Ask anything

ChatGPT

Upgrade

Share

Your API returns the updated task:

JavaScript

```
{
  id: 5,
  title: "New title",
  description: "New description",
  status: "Complete"
}
```

Then React immediately updates.

However, your current method (saving then fetching again) is completely acceptable, especially for a lab. It is simple and guarantees your UI matches the database.

The only thing I would check for your lab requirements is that your edit route also handles all editable fields:

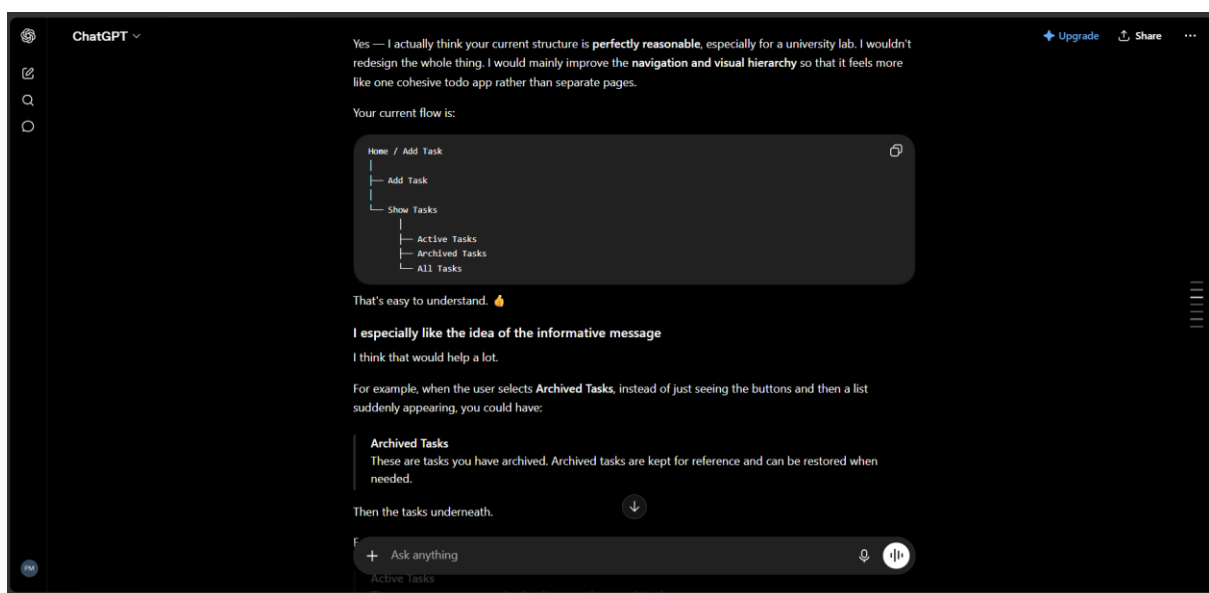
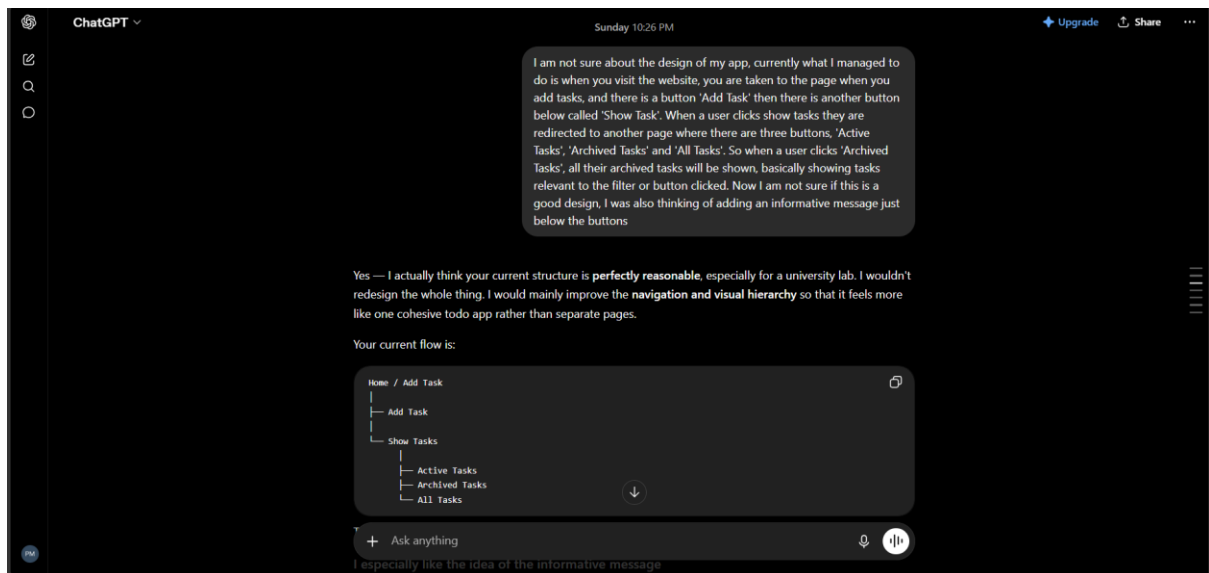
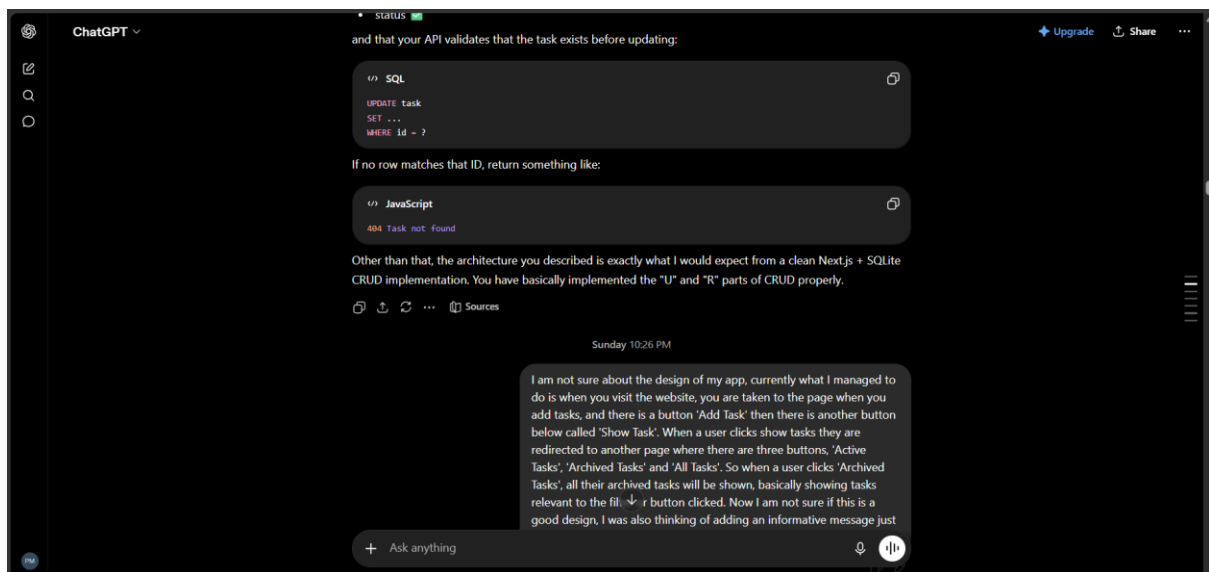
- title
- description
- due date
- topic
- status

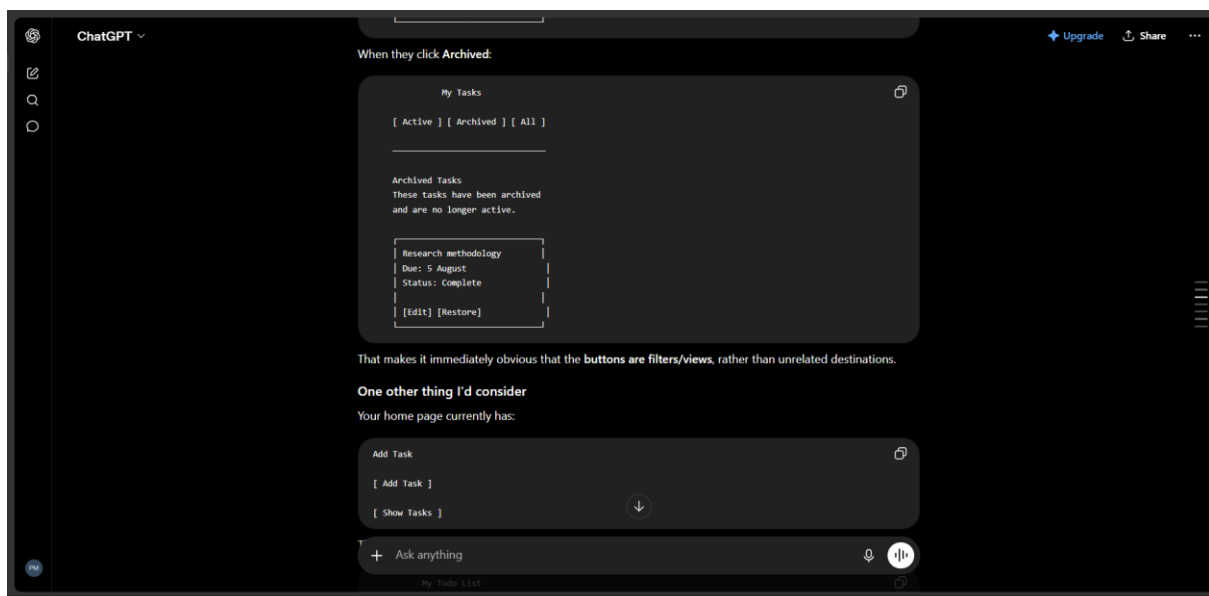
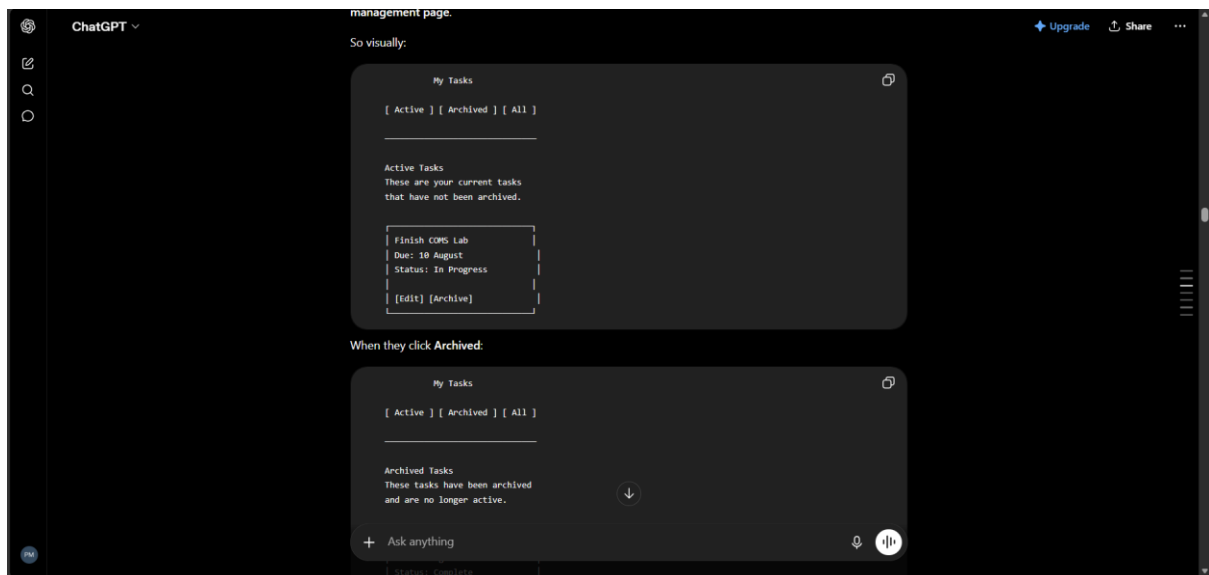
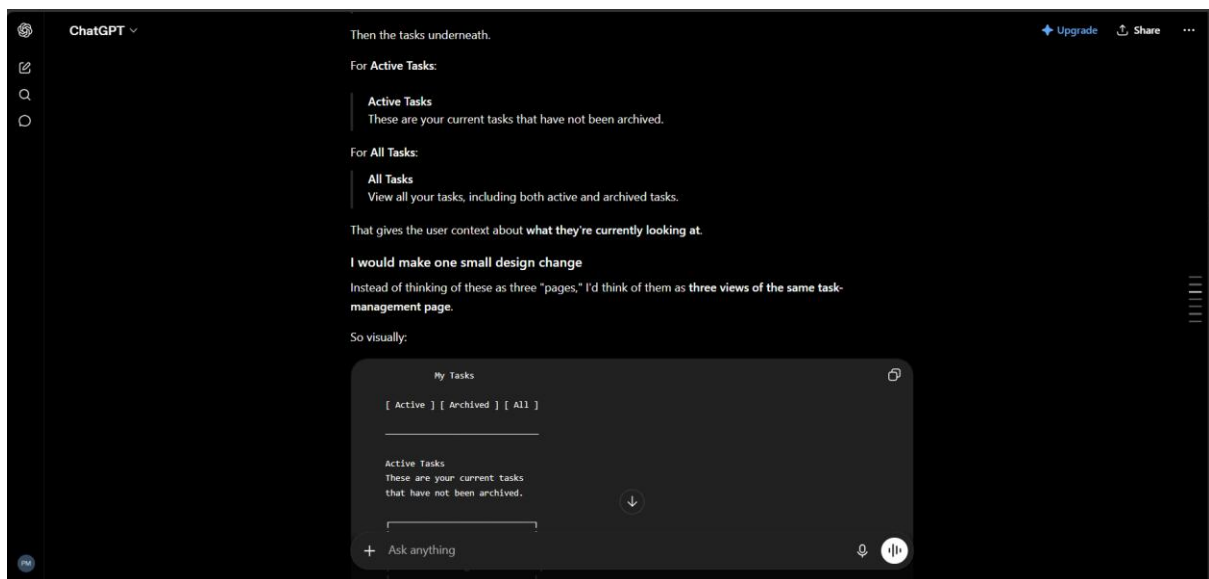
and that your API validates that the task exists before updating:

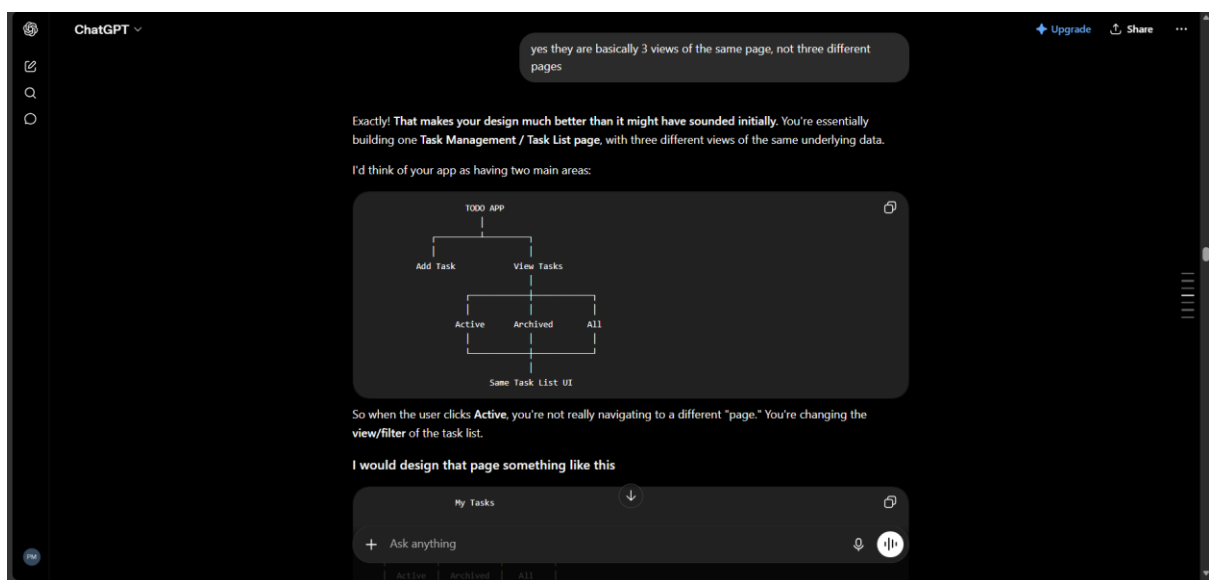
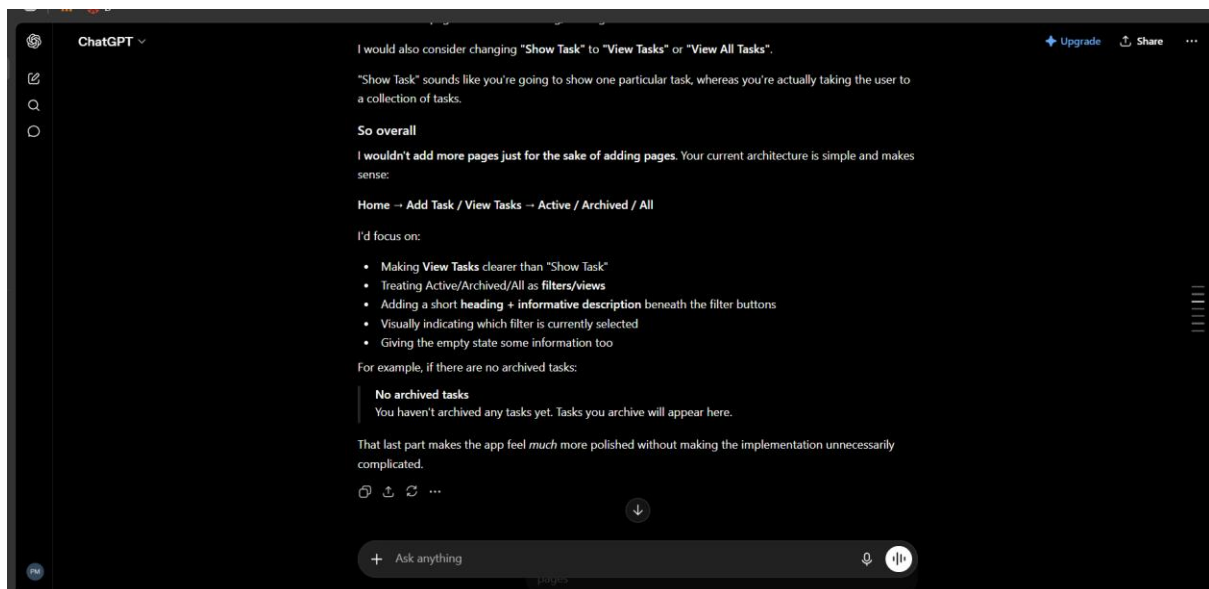
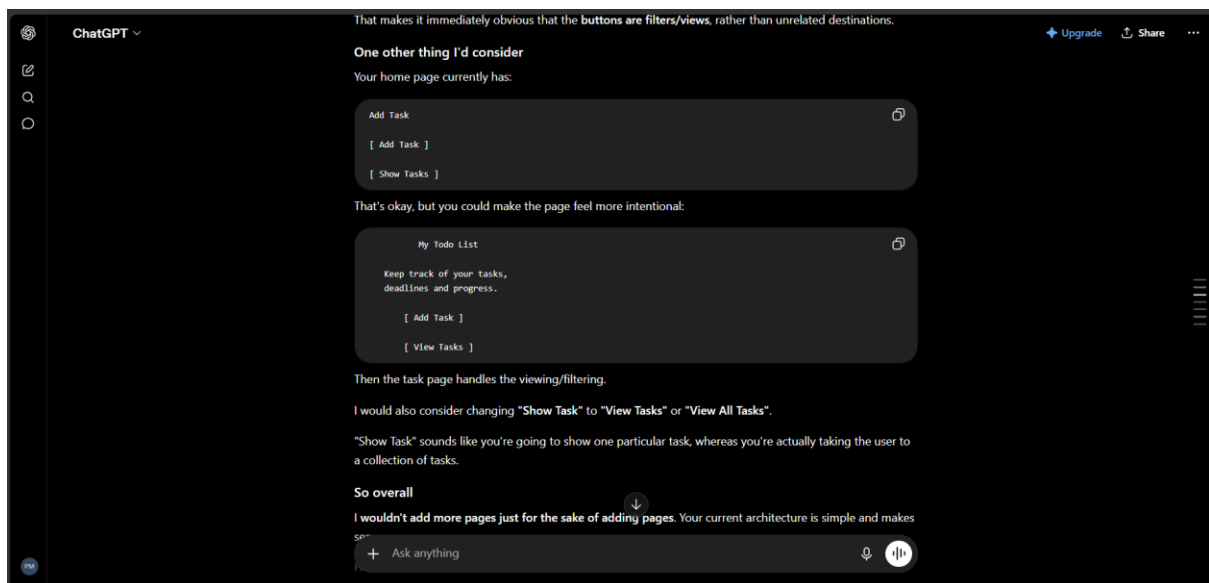
SQL

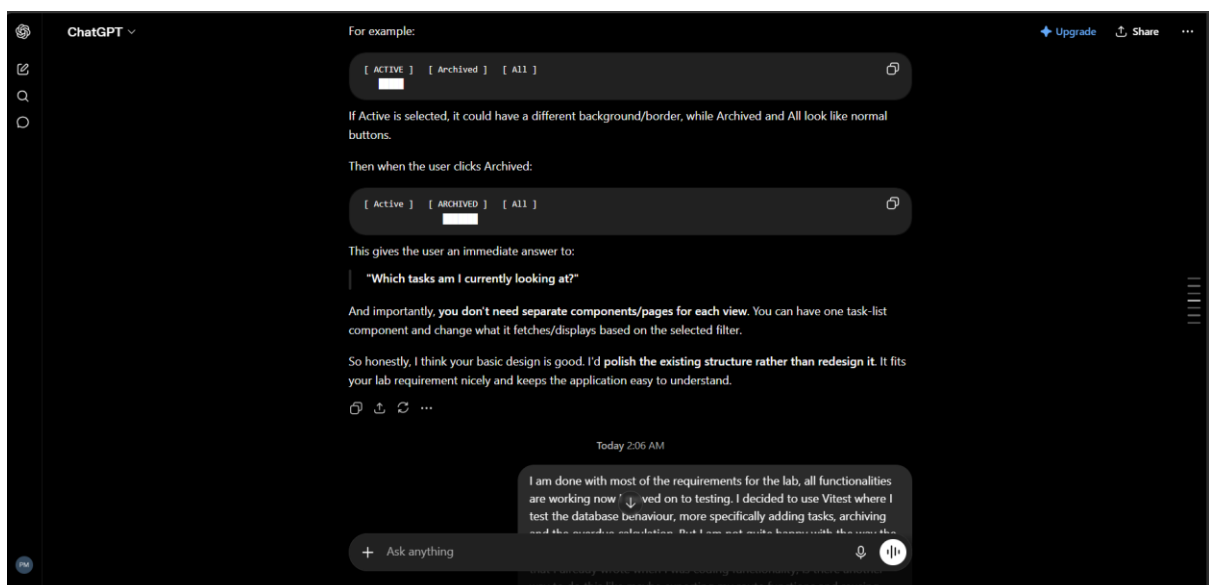
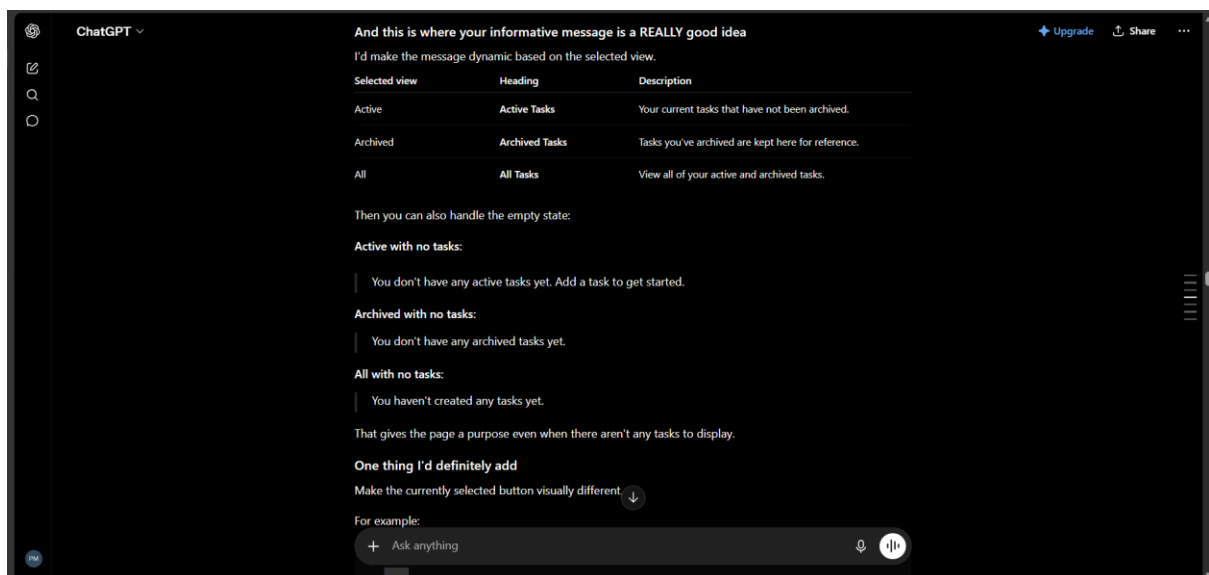
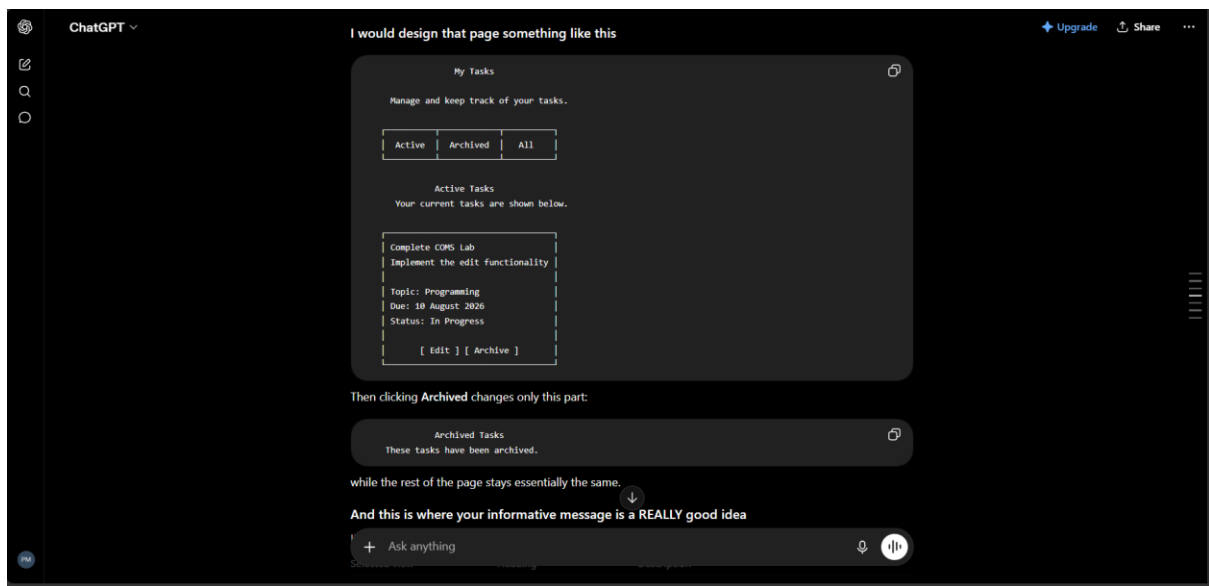
```
UPDATE task
```

+ Ask anything









ChatGPT

Today 2:06 AM

UpgradeShare...

I am done with most of the requirements for the lab, all functionalities are working now I moved on to testing. I decided to use Vitest where I test the database behaviour, more specifically adding tasks, archiving and the overdue calculation. But I am not quite happy with the way the tests are because it appears as though I am rewriting code (well not all) that I already wrote when I was coding functionality, is there another way to do this like maybe exporting my route functions and reusing them. My other concern is that the requirements specifically ask that a throwaway database to be use for tests so won't that mean I will have to modify my functions to take a db as one of the arguments so that when I reuse them in test I am able to use the in-memory SQLite

Show more

This is actually a really good question because you have reached the point where **testing starts influencing the design of your code**, which is exactly what happens in real projects.

Your concern is valid: if your tests are manually recreating the same SQL queries and logic that your routes already use, then you are not really testing your application — you are testing a second implementation of it. That can lead to a situation where:

Your app code:
INSERT INTO tasks ...

Your test code:
INSERT INTO tasks ...

↓

+ Ask anything

The better approach is exactly what you suggested: extract reusable logic and inject the database

ChatGPT

UpgradeShare...

This is actually a really good question because you have reached the point where **testing starts influencing the design of your code**, which is exactly what happens in real projects.

Your concern is valid: if your tests are manually recreating the same SQL queries and logic that your routes already use, then you are not really testing your application — you are testing a second implementation of it. That can lead to a situation where:

Your app code:
INSERT INTO tasks ...

Your test code:
INSERT INTO tasks ...

Both can be wrong in the same way, and your test still passes.

The better approach is exactly what you suggested: **extract reusable logic and inject the database dependency**.

1. Don't test API routes directly for database logic

A common pattern is:

Route
|
↓
Service / Database functions
|
↓
SQLite

↓

+ Ask anything

ChatGPT

UpgradeShare...

1. Don't test API routes directly for database logic

A common pattern is:

Route
|
↓
Service / Database functions
|
↓
SQLite

Currently you might have something like:

app/api/tasks/route.js
|
↓
POST request
|
↓
open database
|
↓
insert task

Instead, separate it:

lib/
├─ db.js
└─ taskService.js
app/api/tasks/route.js
↓

+ Ask anything

taskService.js

ChatGPT

taskService.js

Example:

```
export function createTask(db, task) {
  const stmt = db.prepare(
    `INSERT INTO task
    (title, description, due_date, topic, status)
    VALUES (?, ?, ?, ?, ?)`
  );

  const result = stmt.run(
    task.title,
    task.description,
    task.due_date,
    task.topic,
    task.status
  );

  return result.lastInsertRowid;
}
```

Now this function does not care where the database came from.

It just receives:

```
db
```

+ Ask anything

ChatGPT

It just receives:

```
db
```

and performs the operation.

2. Your API route becomes very thin

Now your route does:

```
import { createTask } from "@lib/taskService";
import db from "@lib/db";

export async function POST(request) {
  const task = await request.json();
  const id = createTask(db, task);
  return Response.json({ id });
}
```

The route's job is only:

- receive HTTP request
- validate input
- call business logic

+ Ask anything

ChatGPT

- call business logic
- return response

3. Your test uses a throwaway database

This is where dependency injection helps.

Your test can create:

```
import Database from "better-sqlite3";

const testDb = new Database(":memory:");
```

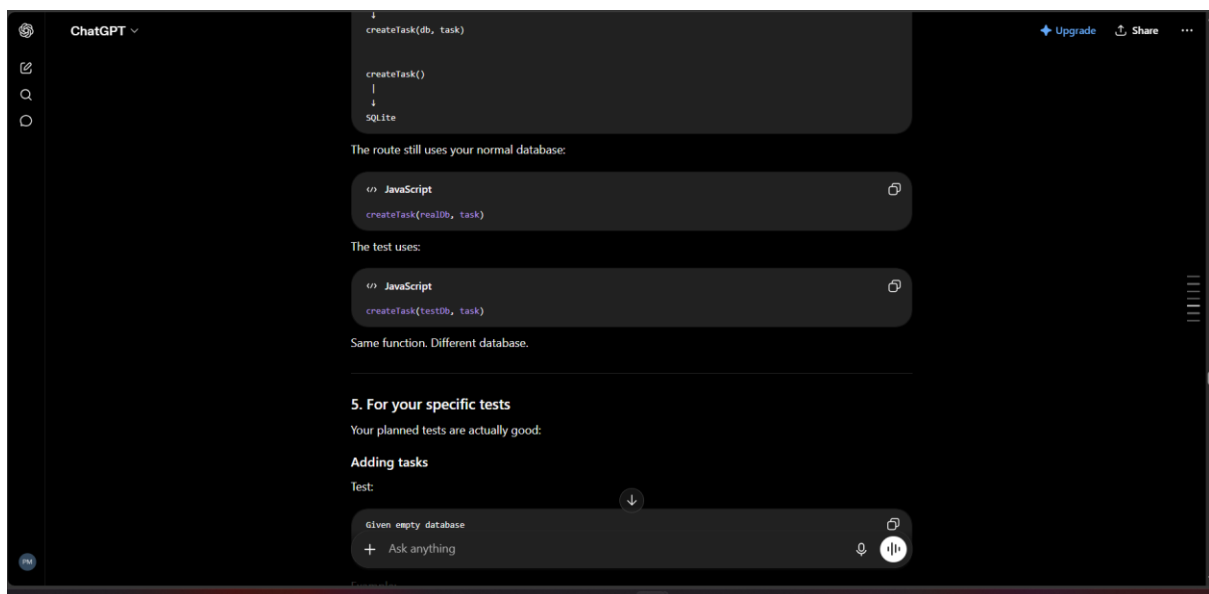
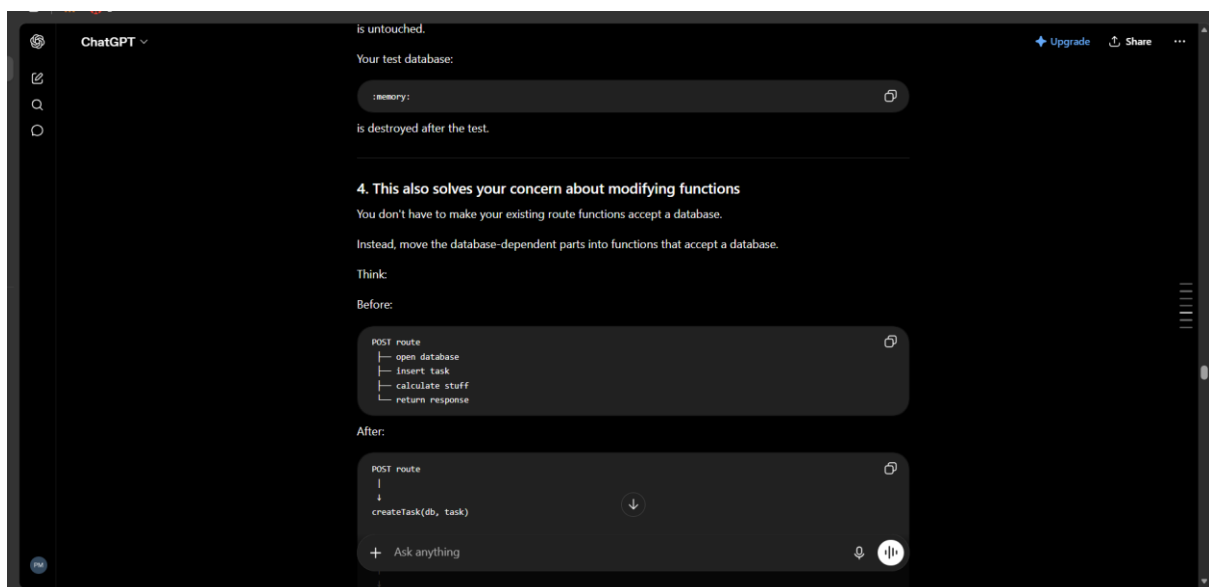
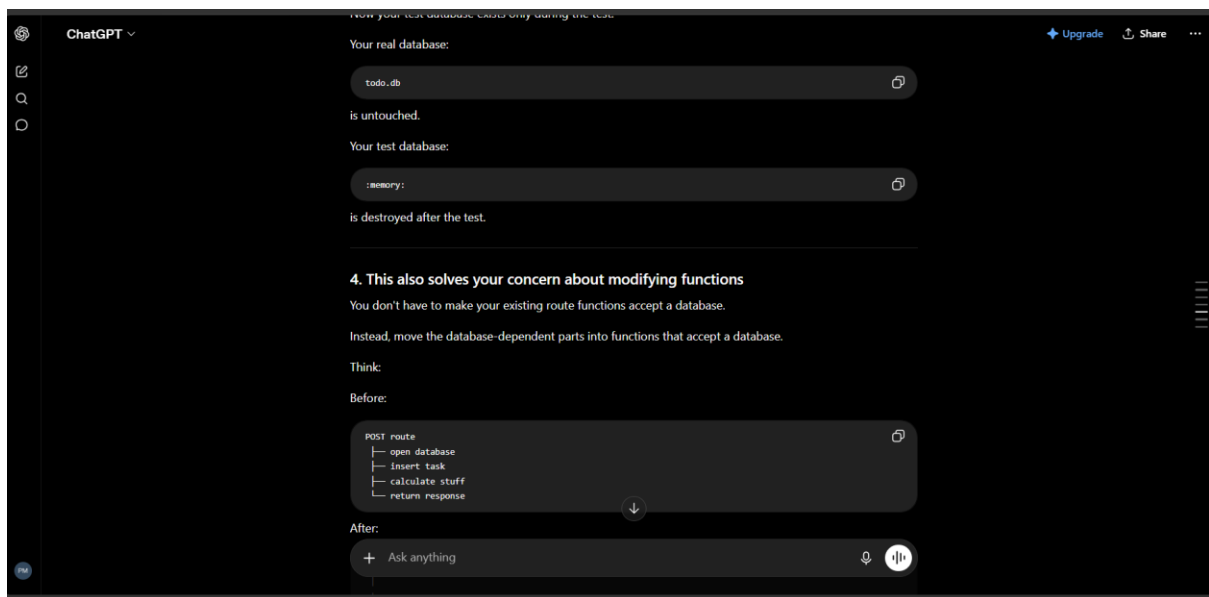
Then:

```
createTask(testDb, {
  title: "Test task",
  description: "Testing",
  due_date: "2026-08-12",
  topic: "Lab",
  status: "Todo"
});
```

Now your test database exists only during the test.

Your real database:

+ Ask anything



⚙️

ChatGPT

✎

🔍

🗨

5. For your specific tests

Upgrade Share

Your planned tests are actually good:

Adding tasks

Test:

Given empty database
When createTask() is called
Then task exists in database

Example:

JavaScript
expect(tasks.length).toBe(1);
expect(tasks[0].title).toBe("Test task");

Archiving

Test:

Given active task
When archiveTask(id)
Then archived = 1

Overdue calculation

+ Ask anything

If you have something like:

⚙️

ChatGPT

✎

🔍

🗨

Overdue calculation

Upgrade Share

This one depends on where your logic is.

If you have something like:

JavaScript
if (due_date < today)
overdue = true

extract that:

JavaScript
export function isOverdue(dueDate) {
return new Date(dueDate) < new Date();
}

Then test:

JavaScript
expect(isOverdue("2020-01-01")).toBe(true);
expect(isOverdue("2030-01-01")).toBe(false);

No database needed.

One thing I would avoid

+ Ask anything

⚙️

ChatGPT

✎

🔍

🗨

One thing I would avoid

Upgrade Share

Don't export your route handlers and call:

JavaScript
POST(request)

inside your tests.

You can do that, but then you are testing:

- HTTP objects
- Next.js behaviour
- request parsing
- responses

instead of your actual task logic.

For a lab, testing the underlying functions is cleaner.

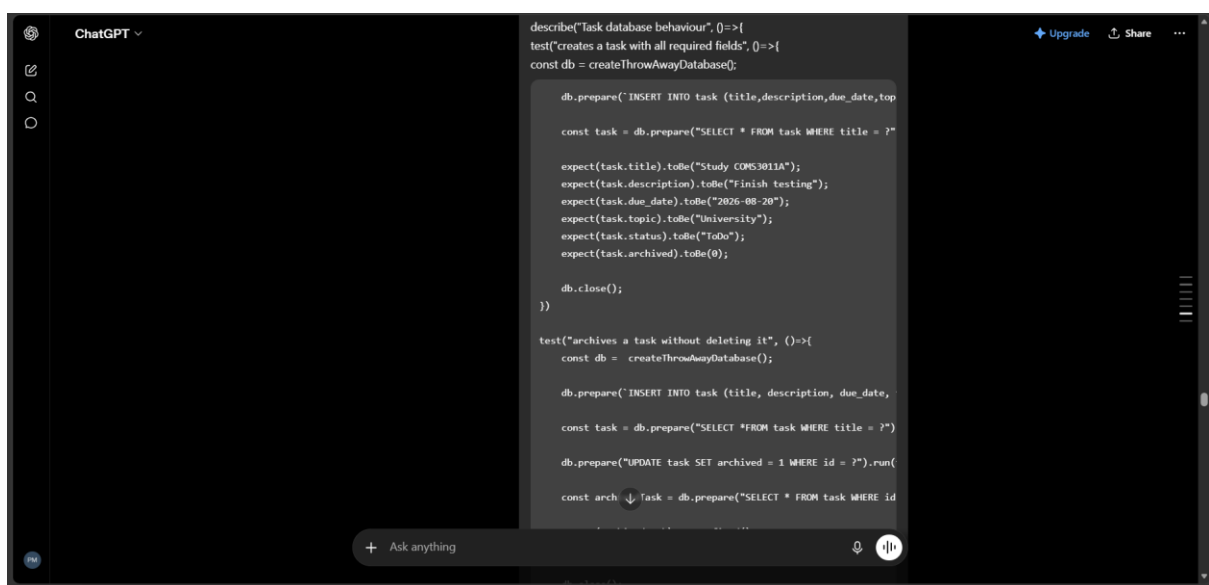
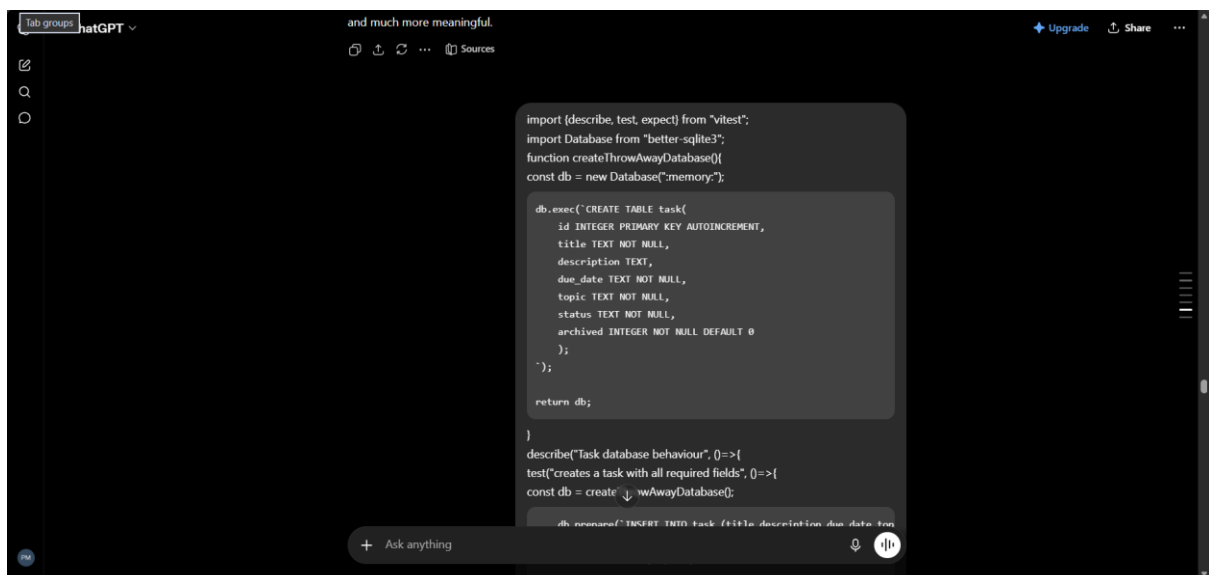
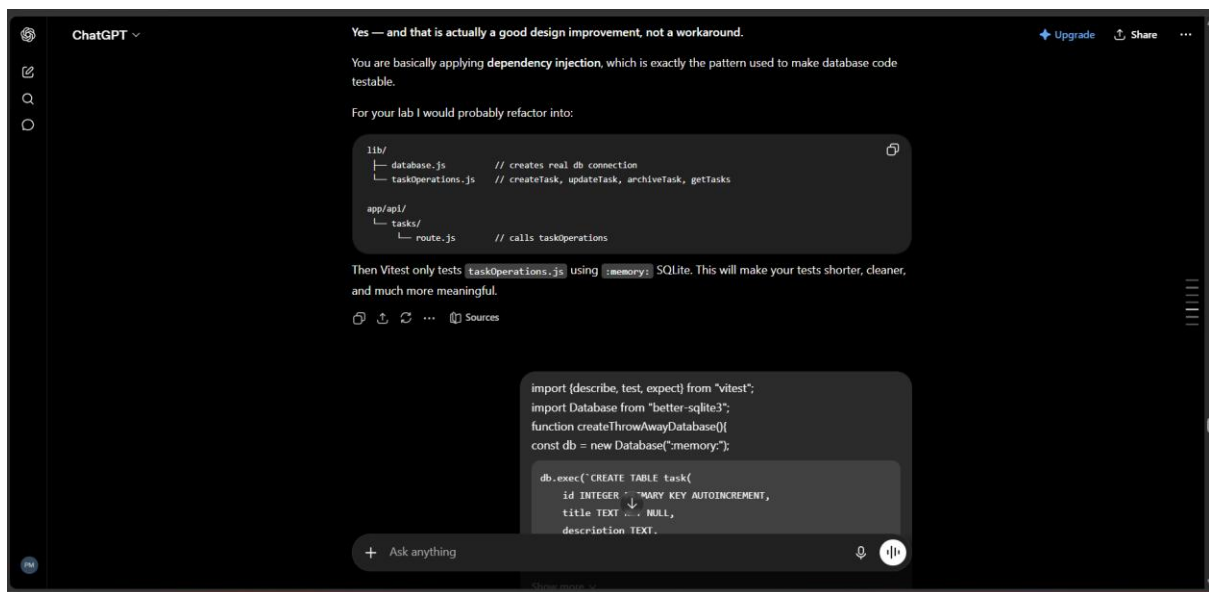
So yes, your instinct is correct:

"won't that mean I will have to modify my functions to take a db as one of the arguments?"

Yes — and that is actually a good design improvement, not a workaround.

You are basically applying dependency injection, which is exactly the pattern used to make database code testable.

+ Ask anything



ChatGPT

```
db.prepare("UPDATE task SET archived = 1 WHERE id = ?").run(
  const archivedTask = db.prepare("SELECT * FROM task WHERE id = ?").get("Old Title");
  expect(archivedTask).toBeDefined();
  expect(archivedTask.archived).toBe(1);
});

db.close();
});

test("updates an existing task", () => {
  const db = createThrowawayDatabase();

  db.prepare(`
    INSERT INTO task
      (title, description, due_date, topic, status)
    VALUES (?, ?, ?, ?, ?)
  `).run(
    "Old Title",
    "Old description",
    "2026-08-20",
    "University",
    "ToDo"
  );

  const task = db.prepare(
    "SELECT * FROM task WHERE title = ?"
  ).get("Old Title");
  expect(task).toBeDefined();
  expect(task.title).toBe("Old Title");
  expect(task.description).toBe("Old description");
  expect(task.due_date).toBe("2026-08-20");
  expect(task.topic).toBe("University");
  expect(task.status).toBe("ToDo");
});

db.close();
});

test("identifies overdue tasks", () => {
  const db = createThrowawayDatabase();

  db.prepare(`
    INSERT INTO task
      (title, description, due_date, topic, status)
    VALUES (?, ?, ?, ?, ?)
  `).run(
    "Overdue Task",
    "Past deadline",
    "2026-08-01",
    "University",
    "ToDo"
  );

  const task = db.prepare(
    "SELECT * FROM task WHERE title = ?"
  ).get("Overdue Task");

  const today = new Date("2026-08-11");
  const dueDate = new Date(task.due_date);

  const days = (dueDate - today) / 86400000;

  expect(days).toBeLessThan(0);
});

db.close();
});
```

Upgrade

Share

+ Ask anything

ChatGPT

```
db.prepare(`
  UPDATE task
  SET title = ?, status = ?
  WHERE id = ?
`).run(
  "Updated Title",
  "Complete",
  task.id
);

const updatedTask = db.prepare(
  "SELECT * FROM task WHERE id = ?"
).get(task.id);

expect(updatedTask.title).toBe("Updated Title");
expect(updatedTask.status).toBe("Complete");

db.close();
});

test("identifies overdue tasks", () => {
  const db = createThrowawayDatabase();

  db.prepare(`
    INSERT INTO task
      (title, description, due_date, topic, status)
    VALUES (?, ?, ?, ?, ?)
  `).run(
    "Overdue Task",
    "Past deadline",
    "2026-08-01",
    "University",
    "ToDo"
  );

  const task = db.prepare(
    "SELECT * FROM task WHERE title = ?"
  ).get("Overdue Task");

  const today = new Date("2026-08-11");
  const dueDate = new Date(task.due_date);

  const days = (dueDate - today) / 86400000;

  expect(days).toBeLessThan(0);
});

db.close();
});
```

Upgrade

Share

+ Ask anything

ChatGPT

```
db.prepare(`
  INSERT INTO task
    (title, description, due_date, topic, status)
  VALUES (?, ?, ?, ?, ?)
`).run(
  "Overdue Task",
  "Past deadline",
  "2026-08-01",
  "University",
  "ToDo"
);

const task = db.prepare(
  "SELECT * FROM task WHERE title = ?"
).get("Overdue Task");

const today = new Date("2026-08-11");
const dueDate = new Date(task.due_date);

const days = (dueDate - today) / 86400000;

expect(days).toBeLessThan(0);

db.close();
});
```

Upgrade

Share

+ Ask anything

